오브젝트 스터디 · 1주차

1장. 객체, 설계

고현석

전체 요약

좋은 설계는
오늘 동작하면서
내일 쉽게 바꿀 수 있는 코드

수단: 캡슐화 · 높은 응집도

핵심 내용

좋은 모듈의 세 가지 조건

1

제대로 동작한다

2

바꾸기 쉽다

3

이해하기 쉽다

핵심 내용

핵심 용어

용어	한 줄 정의
의존성	한 객체가 다른 객체를 사용하는 관계
캡슐화	내부는 감추고 인터페이스만 공개
결합도	모듈끼리 서로 의존하는 정도 → 낮게
응집도	모듈 안의 요소가 연관된 정도 → 높게

핵심 내용

책임의 이동: Theater.enter()

step01 · Theater가 모든 걸 직접 한다

```
if (audience.getBag().hasInvitation()) {
  Ticket ticket = ticketSeller.getTicketOffice().getTicket();
  audience.getBag().setTicket(ticket);
} else {
  Ticket ticket = ticketSeller.getTicketOffice().getTicket();
  audience.getBag().minusAmount(ticket.getFee());
  ticketSeller.getTicketOffice().plusAmount(ticket.getFee());
  audience.getBag().setTicket(ticket);
}
```

step03 · 판매원에게 맡긴다

```
ticketSeller.sellTo(audience);
```

핵심 내용

설계는 트레이드오프다

매표소의 자율성을 높이자

TicketOffice가 Audience를 알게 됐다 (의존성 +1)

내 생각

장점만 보던 시각에서

- 이득이 커도 손해를 감당 못 하면 고르면 안 된다
- 설계에 정답은 없다. 맥락에 맞게 고른다
- "결합도 낮게, 응집도 높게"도 무조건은 아니다

궁금했던 것

Q1. Ticket 같은 작은 클래스도 모듈인가?

→ 그렇다. 모듈은 크기가 아니라 보는 높이에 따라 달라지는 단위

Q2. 서로 전혀 모르고 협력할 수 있나?

→ 없다. 메시지를 보내려면 상대가 받는다는 건 알아야 한다. 최소한만 남긴다

Q3. 데이터와 처리를 묶을지 나눌지 어떻게 비교하나?

→ 일어날 변경을 적고, 고쳐야 하는 클래스 수를 센다

리팩토링할 수 있는 것

step04 → step05에서 바꾼 것

변경	분류
티켓 요금을 생성자로 받기	리팩토링
Bag.hold → receive 이름 변경	리팩토링
얼마 낼지 · 내기 · 받기로 단계 분리	리팩토링
잔액이 부족하면 예외	기능 추가
구매 실패해도 매표소 티켓 유지	버그 수정

리팩토링할 수 있는 것

receive: 공개 메서드는 흐름만

```
public Long receive(Ticket ticket) {
  Long paid = paidFor(ticket);  // 얼마 낼지
  pay(paid);                    // 잔액 검사 후 지불
  setTicket(ticket);            // 받기
  return paid;
}
```

검사 먼저, 상태 바꾸기는 나중에

리팩토링할 수 있는 것

테스트가 잡은 버그: 사라지는 티켓

전 · 티켓을 먼저 뺀다

```
plusAmount(
  audience.buy(getTicket())
);
```

후 · 팔린 뒤에 뺀다

```
Long paid = audience.buy(tickets.get(0));
tickets.remove(0);
plusAmount(paid);
```

돈이 부족한 관람객이 실패하면, 다음 관람객이 티켓을 못 받았다